S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: PRANAV PAUNIKAR
USN:CM24032
Semester / Year:IV/II
Academic Session: 2025-26
Date of Performance:
Date of Submission:

- **Aim:** Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

- **Objectives:**
  1. Implement the Banker's Algorithm to manage resource allocation and prevent deadlocks.
  2. Calculate the current resource need for each process based on allocated and maximum resources.
  3. Determine a safe execution sequence for processes if resources are available.

- **Requirements:**
  - ✓ **Hardware Requirements:**
    - Processor: Minimum 1 GHz
    - RAM: 512 MB or higher
    - Storage: 100 MB free space
  - ✓ **Software Requirements:**
    - Operating System: Linux/Unix-based
    - Shell: Bash 4.0 or higher
    - Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

Banker's Algorithm in Operating System

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

- The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.
- It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.
- By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order.

Why Banker's Algorithm is Named So?

The banker's algorithm is named so because it is used in the banking system to check whether a loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S. Let us assume that the bank has a certain amount of money Y. If a person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

It also helps the OS to successfully share the resources between all the processes. The banker's algorithm uses the notation of a safe allocation state to ensure that granting a resource request cannot lead to a deadlock either immediately or in the future. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in a safe state always.

Components of the Banker's Algorithm

The following **Data structures** are used to implement the Banker's Algorithm:

Let **'n'** be the number of processes in the system and **'m'** be the number of resource types.

1. **Available**
   - It is a 1-D array of size **'m'** indicating the number of available resources of each type.
   - Available[j] = k means there are **'k'** instances of resource type **Rj**
2. **Max**
   - It is a 2-d array of size '**n*m**' that defines the maximum demand of each process in a system.
   - Max[i, j] = k means process **Pi** may request at most **'k'** instances of resource type **Rj.**
3. **Allocation**
   - It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process.
   - Allocation[i, j] = k means process **Pi** is currently allocated **'k'** instances of resource type **Rj**
4. **Need**
   - It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process.
   - Need [i, j] = k means process **Pi** currently needs **'k'** instances of resource type **Rj**
   - Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

Banker's Algorithm

1. Active := *Running U Blocked*
for k = 1 to r:
New_request[k] := Requested_resources[requesting_process, k]
2. Simulated_allocation := *Allocated_resources*
for k = 1 to r: // Compute projected allocation state
Simulated_allocation[requesting_process, k] :=
Simulated_allocation[requesting_process, k] + New_request[k]
3. Feasible := *true*
for k = 1 to r: // Check if projected allocation state is feasible
if Total_resources[k] < Simulated_total_alloc[k]:
Feasible := false
4. if Feasible = *true:*
while set Active contains a process P1 such that:
for all k, Total_resources[k] – Simulated_total_alloc[k] >=
Max_need[P1, k] – Simulated_allocation[P1, k]:
Delete P1 from Active
for k = 1 to r:
Simulated_total_alloc[k] :=
Simulated_total_alloc[k] – Simulated_allocation[P1, k]
5. if set Active is empty:
for k = 1 to r: // Delete the request from pending requests
Requested_resources[requesting_process, k] := 0
for k = 1 to r: // Grant the request
Allocated_resources[requesting_process, k] :=
Allocated_resources[requesting_process, k] + New_request[k]
Total_alloc[k] := Total_alloc[k] + New_request[k]

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length 'm' and 'n' respectively.
   Initialize: Work = Available
   Finish[i] = false; for i=1, 2, 3, 4….n
2. Find an i such that both
   2.1 Finish[i] = false
   2.2 Needi <= Work
   if no such i exists goto step (4)
3. Work = Work + Allocation[i]
   Finish[i] = true
   goto step (2)
4. if Finish [i] = true for all i then the system is in a safe state.

Resource-Request Algorithm

Let Requesti be the request array for process Pi. Requesti [j] = k means process Pi wants k instances of resource type Rj. When a request for resources is made by process Pi, the following actions are taken:

1. If Requesti <= Needi

Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.

2. If Requesti <= Available

Goto step (3); otherwise, Pi must wait, since the resources are not available.

3. Have the system pretend to have allocated the requested resources to process Pi by modifying the state as

follows:

Available = Available – Requesti

Allocationi = Allocationi + Requesti

Needi = Needi– Requesti

Example: Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

- **What will be the content of the Need matrix?**

Need [i, j] = Max [i, j] – Allocation [i, j]

So, the content of Need Matrix is:

- **Is the system in a safe state? If Yes, then what is the safe sequence?**

Applying the Safety algorithm on the given system,

- **What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?**

Unsafe State in The Bankers Algorithm

In the context of the Banker's Algorithm, an **unsafe state** refers to a situation in which, all processes in a system currently hold resources within their maximum needs, there is no guarantee that the system can avoid a deadlock in the future. This state is contrasted with a **safe state**, where there exists a sequence of resource allocation and deallocation that guarantees that all processes can complete without leading to a deadlock.

Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

Example of Unsafe State

Consider a system with three processes (P1, P2, P3) and 6 instances of a resource. Let's say:

- The available instances of the resource are: 1
- The current allocation of resources to processes is:
  - P1: 2
  - P2: 3
  - P3: 1
- The maximum demand (maximum resources each process may eventually request) is:
  - P1: 4
  - P2: 5
  - P3: 3

In situation:

Need = Maximum Demand – Allocation

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- P1 may need 2 more resources to complete.
- P2 may need 2 more resource to complete.
- P3 may need 2 more resources to complete.

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

❖ CODE

```
#include <stdio.h>
#include <stdbool.h>   // For using bool data type

// Function prototypes for better code organization
void printMatrix(const char* name, int matrix[5][4], int rows, int cols);
void printArray(const char* name, int array[], int size, const char* separator);
bool compareNeedWithAvailable(int need[], int available[], int resources);
void printProcessDetails(int process, int need[], int allocation[], int availableBefore[], int
resources);
void printHorizontalLine(int length, char symbol);

int main()
{
    // Declare loop variables
    int i, j, k;

    // System Configuration
    int processes = 5;  // Number of processes in the system
    int resources = 4;  // Number of resource types (R1, R2, R3, R4)

    // Current allocation matrix: resources currently held by each process
    // Format: allocation[process_id][resource_type]
    int allocation[5][4] = {
        {0, 0, 1, 4},  // Process P0 has: R3=1, R4=4
        {0, 6, 3, 2},  // Process P1 has: R2=6, R3=3, R4=2
        {0, 0, 1, 2},  // Process P2 has: R3=1, R4=2
        {1, 0, 0, 0},  // Process P3 has: R1=1
        {1, 3, 5, 4}   // Process P4 has: R1=1, R2=3, R3=5, R4=4
    };

    // Maximum demand matrix: maximum resources each process may need
    int max[5][4] = {
        {0, 6, 5, 6},  // Process P0 may need up to: R2=6, R3=5, R4=6
        {0, 6, 5, 2},  // Process P1 may need up to: R2=6, R3=5, R4=2
        {0, 0, 1, 2},  // Process P2 may need up to: R3=1, R4=2
        {1, 7, 5, 0},  // Process P3 may need up to: R1=1, R2=7, R3=5
```

```
        {2, 3, 5, 6}   // Process P4 may need up to: R1=2, R2=3, R3=5, R4=6
    };

    // Initially available resources (total system resources minus allocated ones)
    int available[4] = {1, 6, 2, 0};  // R1=1, R2=6, R3=2, R4=0 initially available

    // Data structures for Banker's algorithm
    int need[5][4];          // Need matrix (Max - Allocation)
    int finish[5] = {0};     // Track finished processes (0 = not finished, 1 = finished)
    int safeSeq[5];          // Store the safe sequence found
    int count = 0;           // Number of processes in safe sequence
    bool isSafe = true;      // Flag to track if system is in safe state

    // Print header for better visualization
    printf("\n");
    printHorizontalLine(60, '=');
    printf("          BANKER'S ALGORITHM - DEADLOCK AVOIDANCE\n");
    printHorizontalLine(60, '=');
    printf("\n");

    // Display initial system state
    printf("INITIAL SYSTEM STATE:\n");
    printHorizontalLine(60, '-');
    printMatrix("ALLOCATION MATRIX (Current Resources Held)", allocation, processes,
resources);
    printMatrix("MAXIMUM DEMAND MATRIX (Maximum Resources Needed)", max,
processes, resources);

    // Calculate NEED matrix (MAX - ALLOCATION)
    printf("\nCALCULATING NEED MATRIX (MAX - ALLOCATION):\n");
    printHorizontalLine(60, '-');
    for(i = 0; i < processes; i++)
    {
        for(j = 0; j < resources; j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
        }
```

```
    }
    printMatrix("NEED MATRIX (Remaining Resources Needed)", need, processes,
resources);

    // Display initial available resources
    printf("\nINITIAL AVAILABLE RESOURCES: ");
    printArray("", available, resources, " ");
    printf("\n");

    // Start Banker's Algorithm to find safe sequence
    printf("\n%s", "EXECUTING BANKER'S ALGORITHM:");
    printf("\n%s", "==============================");
    printf("\n\n%s", "Searching for safe sequence...\n");

    // Main loop: continue until all processes are finished or no safe sequence exists
    while(count < processes && isSafe)
    {
        bool found = false;  // Flag to check if any process was allocated in this iteration

        // Try to find a process that can be executed with current available resources
        for(i = 0; i < processes; i++)
        {
            // Check if process i is not finished yet
            if(finish[i] == 0)
            {
                // Check if process i's needs can be satisfied with current available resources
                if(compareNeedWithAvailable(need[i], available, resources))
                {
                    found = true;  // Found a process that can be executed
                    safeSeq[count] = i;  // Add to safe sequence
                    finish[i] = 1;  // Mark as finished

                    // Print detailed information about process execution
                    printProcessDetails(i, need[i], allocation[i], available, resources);

                    // Update available resources by adding the allocated resources back
                    for(k = 0; k < resources; k++)
```

```
                {
                    available[k] += allocation[i][k];
                }

                // Display updated available resources
                printf("  --> New Available Resources: ");
                printArray("", available, resources, " ");
                printf("\n");

                count++;  // Increment finished process count

                // Break to restart scanning from beginning (as per algorithm)
                // This ensures we always check from P0 after each allocation
                break;
            }
        }
    }

    // If no process could be allocated in this iteration, system is unsafe
    if(!found)
    {
        isSafe = false;
        printf("\n⚠  DEADLOCK WARNING: No process can be allocated with current
resources!\n");
    }
  }

  // Display final results
  printf("\n");
  printHorizontalLine(60, '=');
  printf("                    FINAL RESULTS\n");
  printHorizontalLine(60, '=');

  if(isSafe && count == processes)
  {
    printf("\nSAFE SEQUENCE FOUND:\n");
    printf("   ");
```

```
for(i = 0; i < processes; i++)
{
    printf("P%d", safeSeq[i]);
    if(i < processes - 1)
        printf(" → ");
}
printf("\n\n");

// Display final available resources
printf("FINAL AVAILABLE RESOURCES: ");
printArray("", available, resources, " ");
printf("\n");

// Display process completion status
printf("\nPROCESS COMPLETION STATUS:\n");
for(i = 0; i < processes; i++)
{
    printf("  P%d: %s\n", i, finish[i] ? "completed" : "Not Completed");
}

// Calculate total resources in system
printf("\nTOTAL SYSTEM RESOURCES:\n");
int total[4] = {0};
for(j = 0; j < resources; j++)
{
    total[j] = available[j];  // Start with available
    for(i = 0; i < processes; i++)
    {
        total[j] += allocation[i][j];  // Add all allocated resources
    }
}
printf("  Resource Types: ");
for(j = 0; j < resources; j++)
{
    printf("R%d=%d ", j+1, total[j]);
}
printf("\n");
```

```
    }
    else
    {
        printf("\nNO SAFE SEQUENCE EXISTS!\n");
        printf("   The system is in an UNSAFE state and deadlock may occur.\n\n");

        // Display which processes couldn't be completed
        printf("PROCESSES THAT COULDN'T BE COMPLETED:\n");
        for(i = 0; i < processes; i++)
        {
            if(finish[i] == 0)
            {
                printf("  P%d - Need: ", i);
                printArray("", need[i], resources, " ");
                printf(" (cannot be satisfied with available: ");
                printArray("", available, resources, " ");
                printf(")\n");
            }
        }
    }

    printHorizontalLine(60, '=');
    printf("\n");

    return 0;
}
void printMatrix(const char* name, int matrix[5][4], int rows, int cols)
{
    int i, j;
    printf("\n%s:\n", name);
    printf("      ");
    for(j = 0; j < cols; j++)
    {
        printf("R%d ", j+1);
    }
    printf("\n");
    printf("      ");
```

```
    for(j = 0; j < cols; j++)
    {
        printf("---");
    }
    printf("\n");

    for(i = 0; i < rows; i++)
    {
        printf("P%d | ", i);
        for(j = 0; j < cols; j++)
        {
            printf("%d  ", matrix[i][j]);
        }
        printf("\n");
    }
}
void printArray(const char* name, int array[], int size, const char* separator)
{
    int i;
    if(name[0] != '\0')
        printf("%s: ", name);

    printf("[");
    for(i = 0; i < size; i++)
    {
        printf("%d", array[i]);
        if(i < size - 1)
            printf("%s", separator);
    }
    printf("]");
}

bool compareNeedWithAvailable(int need[], int available[], int resources)
{
    int j;
    for(j = 0; j < resources; j++)
    {
```

```
        if(need[j] > available[j])
            return false;  // Need exceeds available
    }
    return true;  // All needs can be satisfied
}

/**
 * Function to print detailed process execution information
 */
void printProcessDetails(int process, int need[], int allocation[], int availableBefore[], int
resources)
{
    printf("\n ▶ Process P%d selected for execution:\n", process);
    printf("    Current Need      : ");
    printArray("", need, resources, " ");
    printf("\n");
    printf("    Current Allocation: ");
    printArray("", allocation, resources, " ");
    printf("\n");
    printf("    Available Before  : ");
    printArray("", availableBefore, resources, " ");
}

/**
 * Function to print a horizontal line with given character
 */
void printHorizontalLine(int length, char symbol)
{
    int i;
    for(i = 0; i < length; i++)
    {
        printf("%c", symbol);
    }
    printf("\n");
}
```

❖ Output:

```
INITIAL SYSTEM STATE:
------------------------------------------------------------

ALLOCATION MATRIX (Current Resources Held):
      R1 R2 R3 R4
      ------------
P0 | 0  0  1  4
P1 | 0  6  3  2
P2 | 0  0  1  2
P3 | 1  0  0  0
P4 | 1  3  5  4

MAXIMUM DEMAND MATRIX (Maximum Resources Needed):
      R1 R2 R3 R4
      ------------
P0 | 0  6  5  6
P1 | 0  6  5  2
P2 | 0  0  1  2
P3 | 1  7  5  0
P4 | 2  3  5  6

CALCULATING NEED MATRIX (MAX - ALLOCATION):
------------------------------------------------------------

NEED MATRIX (Remaining Resources Needed):
      R1 R2 R3 R4
      ------------
P0 | 0  6  4  2
P1 | 0  0  2  0
P2 | 0  0  0  0
P3 | 0  7  5  0
P4 | 1  0  0  2

INITIAL AVAILABLE RESOURCES: [1 6 2 0]

EXECUTING BANKER'S ALGORITHM:
================================

Searching for safe sequence...

  ▶ Process P1 selected for execution:
    Current Need      : [0 0 2 0]
    Current Allocation: [0 6 3 2]
    Available Before  : [1 6 2 0]  --> New Available Resources: [1 12 5 2]

  ▶ Process P0 selected for execution:
    Current Need      : [0 6 4 2]
    Current Allocation: [0 0 1 4]
    Available Before  : [1 12 5 2]  --> New Available Resources: [1 12 6 6]

  ▶ Process P2 selected for execution:
    Current Need      : [0 0 0 0]
    Current Allocation: [0 0 1 2]
    Available Before  : [1 12 6 6]  --> New Available Resources: [1 12 7 8]

  ▶ Process P3 selected for execution:
    Current Need      : [0 7 5 0]
    Current Allocation: [1 0 0 0]
    Available Before  : [1 12 7 8]  --> New Available Resources: [2 12 7 8]

  ▶ Process P4 selected for execution:
    Current Need      : [1 0 0 2]
    Current Allocation: [1 3 5 4]
    Available Before  : [2 12 7 8]  --> New Available Resources: [3 15 12 12]

============================================================
                        FINAL RESULTS
============================================================

SAFE SEQUENCE FOUND:
   P1 → P0 → P2 → P3 → P4

FINAL AVAILABLE RESOURCES: [3 15 12 12]
```

Conclusion: In this practical, we conclude that the Banker's Algorithm effectively ensures safe resource allocation by preventing deadlocks. By calculating the current resource need of each process and checking for a safe execution sequence, we can determine whether all processes can complete execution without system instability

❖ **Discussion Questions:**

1. **What is the purpose of the Banker's Algorithm?**
2. **How do you calculate the current need of each process?**
3. **What are the conditions for a system to be in a safe state?**
4. **What happens if no safe sequence exists in the Banker's Algorithm?**.
5. **What are the key inputs required to implement the Banker's Algorithm?**

❖ **References:**

https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/

https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26